Hypergraph-Aware Mapping of Seizure-Detecting Spiking Neural Networks onto Neuromorphic Cores

Rishav Singh
Department of Electrical and Electronics Engineering
Dr. Vishwanath Karad MIT World Peace University
Pune, India

Dr. Sarika Patil
Assistant Professor
Department of Electrical and Electronics Engineering
Dr. Vishwanath Karad MIT World Peace University
Pune, India

***Abstract*—Spiking neural networks (SNNs) can run on neuromorphic chips with very little energy. But a trained network cannot just be dropped onto a chip. Its neurons must first be split into groups, and each group must be placed on a core. A poor split sends many spikes between cores, and that costs energy and time. In this work we study this mapping step for an SNN that detects seizures in EEG. We describe the network as a hypergraph, where each neuron's output is one hyperedge, and we use measured firing rates as weights. We compare a hypergraph-aware pipeline with a naive layer-order pipeline on five SNNs that passed a fixed accuracy check on CHB-MIT patient chb01. Under a Loihi-based cost model, the hypergraph-aware pipeline lowered the estimated energy-latency product by 23 to 73 percent (five-seed means). These are model estimates, not measurements on a chip. An ablation shows that most of this gain comes from traffic-aware core placement. Hypergraph partitioning adds a clear extra gain only on the large sparse network, about 11 percent. We then repeated the study on 22 more CHB-MIT patients and on one model trained on pooled data, 144 models in total. Here the accuracy check was not used as a filter, only as a check. The full pipeline beat the naive one in all 138 patient models, by 15 to 79 percent. Partitioning added a gain only on the large pruned networks (median 9.3 percent, positive in all 23 patients, and clearly passing our screen in 14 of 23), not on the small ones, and it was mixed on the medium ones. Pruned models also fire less and often lose accuracy, so their lower cost is not a free gain. We then ran four hardware-oriented checks, all in simulation: 8-bit weights kept the AUC unchanged, a core memory audit showed that dense models overflow while 90 percent sparse models fit, a cycle-accurate mesh replay of a seizure window showed fewer router stalls, and a Verilog router model agreed with the Python simulator. To make the work easier to follow, we also built a small web dashboard that replays a recorded EEG hour through one trained model and shows the network, the chip, and the traffic. It is a visualisation of what we did, not a finished product. We also report the ideas that failed on the way, such as grouping EEG channels into hyperedges before encoding and aiming at a wearable chip, and we explain what each result taught us and how it changed our next step.**

***Index Terms*—spiking neural networks, neuromorphic hardware, hypergraph partitioning, core mapping, seizure detection, EEG**

I. Introduction

Seizures can be found in scalp EEG, and deep learning models can detect them well [1]. Many of these models need a lot of computing, which can make them hard to run all day on a small device. Spiking neural networks (SNNs) are another option. A neuron in an SNN sends a spike only when something happens, so a chip built for spikes can save energy [2], [3]. How much it saves depends on the network, the task, and the chip. Busia *et al.* [4] showed that a very small SNN on an FPGA can detect seizures in CHB-MIT recordings [5]–[7] at a tiny energy cost per inference.

This project started as an attempt to build on that wearable detector. Our first idea was to group EEG channels with hypergraphs before the spike encoder [8]. Our second idea was to build a small FPGA system around the result. Neither idea survived testing, and we describe both in Section IV. What was left was a more basic question: once an SNN is trained, how should its neurons be placed on a chip that is made of many small cores?

A neuromorphic chip has many cores. Each core can hold only a limited number of neurons, inbound axons, and synapses. When a spike must go from one core to another, it travels over an on-chip network, and this costs energy and time. So the place where each neuron lives matters. Ronzani and Silvano [9] point out that a neuron sends the same spike to all of its targets. If five targets sit in one core, the chip sends one message and the core copies it locally. A normal graph does not see this and counts five messages. They therefore model the SNN as a hypergraph, with one hyperedge for each sending neuron.

Mapping SNNs onto cores has been studied before. SpiNeMap [10], SNEAP [11], and EdgeMap [12] split the network with graph methods and then place the parts on the chip to cut spike traffic. Ronzani and Silvano have since shown the hypergraph view on large networks, including a GPU version [13]. So mapping with hypergraphs is not a new idea, and we do not claim it is. Our question is narrower. For small SNNs that detect seizures, how much of the gain comes from placement and how much from partitioning, and when does partitioning start to help? We also ask whether what we see on one patient holds on others. Section II-E compares our work with earlier tools in more detail.

We built a mapping pipeline around this idea and tested it on seizure-detecting SNNs of three sizes. We first tested it on one patient (chb01). A result from one patient can be a fluke, so we then repeated the study on 22 more patients and on a pooled model. We did not only look for a win. We also asked where any gain comes from, checked it with hardware-style tests, and kept a record of every idea that failed. A large part

of this paper is about what we concluded at each step and how that changed what we did next.

The main contributions are:

- A hypergraph-aware mapping pipeline for EEG-trained SNNs, compared with a naive layer-order pipeline on five models that passed a pre-declared accuracy gate.
- An ablation that splits the gain into a placement part and a partitioning part, and a check that the main finding repeats on a second set of separately trained models.
- A check on 22 more patients and one pooled model (144 models in total), with the gain split into placement and partitioning, and with the AUC gate used as a check instead of a filter.
- Four hardware-oriented checks: 8-bit quantization, a core memory audit, a cycle-accurate mesh replay of normal and seizure windows, and a Verilog router simulation.
- A small web dashboard that brings the trained model, the two mappings, and the cost model into one page, as a visual way to follow the work. It is a visualisation of what we have done so far, not a finished product.
- A full record of the weak or failed ideas (EEG-side hypergraphs, the wearable target, and early results that we later withdrew), with the conclusions we drew from each.

Section II gives background and compares earlier mapping tools. Section III describes the data and the baseline model. Section IV covers the early ideas that did not hold up. Section V explains the mapping method, and Section VI the models and setup. Section VII gives the results, including the check on more patients, Section VIII shows the dashboard, Section IX the lessons, and Sections X and XI the limits and the conclusion.

II. Background and Related Work

A. SNNs for Seizure Detection

An SNN neuron keeps an internal value that leaks over time. Input spikes raise the value. When the value crosses a threshold, the neuron fires and the value is reduced. We use the standard leaky integrate-and-fire (LIF) neuron [2], [14]. For the first replication we used snnTorch [14], and for the later work we wrote our own PyTorch code.

Busia *et al.* [4] built a seizure detector on a Lattice iCE40UP5K FPGA. It reads four temporal EEG channels. Each sample is turned into one spike on one of 16 amplitude levels, so the input layer has 64 neurons and exactly four of them are active at any step. A 64-8-1 LIF network with 529 weights makes the decision. They report 4.55 nJ and 0.5 μs per inference. We use their encoder and network shape as our starting point. Their results come from their own training and decision rules, so our numbers in Section III-C are not a like-for-like comparison.

B. Mapping SNNs onto Cores

Chips such as Loihi [15] are built from many cores joined by a network-on-chip. Mapping has two steps. *Partitioning* decides which neurons share a core. *Placement* decides where each core sits on the chip grid. Each core has three limits: the number of neurons (C_{npc}), the number of distinct inbound axons (C_{apc}), and the number of synapses (C_{spc}) [9].

Ronzani and Silvano [9] model the SNN as a directed hypergraph. Two ideas follow. *Synaptic reuse* says that neurons which listen to the same senders should share a core, so the core receives each spike once. *Connections locality* says that cores which talk a lot should sit close together on the grid. They also give a greedy partitioner based on hyperedge overlap.

Other mapping tools. Many tools split an SNN into clusters and then place the clusters on the chip. Table I lists the main ones. SpiNeMap and DFSynthesizer [10], [16] use Kernighan–Lin graph partitioning and a particle-swarm placement. SNEAP [11] also partitions and then places on a network-on-chip, to cut spike traffic and hops. EdgeMap [12] uses a flow-based partitioner and a genetic placement. NeuMap [17] computes firing rates from the network and uses them to guide the split of feedforward networks. Jin *et al.* [18] aim at very large layered networks, and SNNcut [19] builds on their placement with a fast one-pass split. Hypergraph partitioners from the general literature [20], such as Mt-KaHyPar [21], are strong, but they do not model the core limits of neuromorphic chips. Limits on tile resources can also be handled by splitting neurons that have many inputs [22]. Ronzani and Silvano have also given a GPU version of hypergraph partitioning and placement [13]. It is the closest work to ours. They report gains over sequential tools on twelve large networks. We did not run it.

C. Trace-Driven Evaluation

Mapping papers such as SpiNeMap [10], SNEAP [11], and EdgeMap [12] do not rely only on formulas. They record real spike traces from the software network and replay them on a cycle-accurate model of the on-chip network. Other work scores mappings with an analytical cost model [9], [18], [19], as we do in Section V-B. We use both ideas: the analytical model for the main results, and a trace replay in Section VII-E.

D. Hypergraphs for EEG

Pisarchik *et al.* [8] build brain hypergraphs from coherence, cross-correlation, and mutual information. Their study is a single-subject test on an attention task, not on seizures. The reason to use hyperedges is that many brain regions can work together at the same time, and a hyperedge can join them all. We borrowed this as our first hypothesis, at the input side of the SNN (Section IV-A). Their work does not show that grouping channels before spike encoding helps seizure detection. That was only our guess.

E. What This Work Adds

Earlier tools already cut spike traffic and hops by splitting the network and placing the parts [10]–[12], [17], [18]. The hypergraph papers [9], [13] add the point that one spike sent to one core should be counted once, and they test it on large networks made for image tasks and brain models. We do not

TABLE I
SOME SNN MAPPING TOOLS AND HOW THEY RELATE TO THIS WORK. THESE ARE SHORT SUMMARIES OF WHAT EACH PAPER DESCRIBES. WE DID NOT RE-RUN ANY OF THESE TOOLS.

Work	Network model	Partitioning	Placement
SpiNeMap [10], DFSynthesizer [16]	graph	Kernighan–Lin, recursive	particle swarm
EdgeMap [12]	graph	streaming, flow-based	genetic algorithm
NeuMap [17]	graph, with firing rates	clusters inside groups of adjacent layers	meta-heuristic search
Jin *et al.* [18]	graph	built for very large layered networks	Hilbert curve, force-directed swaps
SNNcut [19]	graph	one pass in node order	Hilbert curve, force-directed swaps
Ronzani and Silvano [9]	hypergraph	sequential, overlap, hierarchical	Hilbert, spectral, force-directed
Ronzani and Silvano (GPU) [13]	hypergraph	multilevel, on GPU	recursive bisection, then swaps
This work	hypergraph	simple greedy overlap (after [9])	Hilbert curve, 1500 random swaps

TABLE II
DATA SPLIT USED FOR THE FINAL EXPERIMENTS (CHB01).

Set	Records	Windows	Seizure windows
Train	03, 04, 15, 21, 26	2090	40
Validation	16	450	6
Test	18	450	11

TABLE III
REPLICATION ON CHB01 (6 OF 7 FOLDS) AGAINST THE PUBLISHED ROW.

Metric	Ours	Busia *et al.*
Accuracy	0.992	0.994
Sensitivity (segment)	1.000	0.687
Specificity	0.992	1.000
AUC	0.998	0.976
Event sensitivity	1.000	1.000
False positives per hour	1.776	0.000

claim to introduce hypergraphs for SNN mapping, and we did not run the tools above on our models. What we add is a narrower study on a different kind of workload: small SNNs that detect seizures in EEG. We split the gain into a placement part and a partitioning part, we ask at what size and sparsity partitioning starts to help, and we check the result with simple hardware-style tests. A direct comparison with the stronger tools is left for future work.

III. DATA AND BASELINE MODEL

A. Data

We used CHB-MIT [5]–[7], patient chb01 only, sampled at 256 Hz. We took the same four bipolar channels as Busia *et al.*: F7-T7, T7-P7, F8-T8, and T8-P8. One practical problem came up early. The recordings contain two channels named T8-P8 because of a montage change, and the reader library renamed them. We fixed this by always picking the original one. After that, the seizure intervals we parsed added up to 442 s over seven records, which matches the total in Busia *et al.* and told us that channel choice and label parsing were correct.

For the final experiments we cut each record into non-overlapping windows of 2048 steps (8 s). A window is labeled as seizure if at least half of it overlaps a seizure. Files were split by record, not by window, so that neighbouring windows could not leak between sets (Table II). We also checked our labels against the official summary file, and all seven seizure intervals matched.

B. Spike Encoding

Each channel's amplitude range is cut into 16 equal levels. The range comes from the first 300 s of record 03. The level steps came out as 34.5, 33.2, 32.6, and 28.1 μV for the four channels. At every time step, each channel produces one spike on the level where its voltage falls. This gives 64 input neurons and exactly 4.0 input spikes per step, which we checked in the code.

C. Replicating the Wearable Detector

We first rebuilt the 64-8-1 network of Busia *et al.* (529 parameters, including 9 biases) and tested it with leave-one-record-out folds. Training on a laptop was far too slow. A full 500-epoch run over all seven folds was projected at about 58 hours. We moved to a GPU and cut training to 10 epochs per fold. We also dropped the paper's post-ictal filtering and output smoothing. One fold (record 18) gave a dead model whose output never fired, and we left it out of the average and reported it separately. Table III shows the other six folds.

AUC and event sensitivity match or beat the paper, but this should not be read as our model being better. The training, post-processing, threshold rule, and folds are different. False positives are much higher, and this is what we expect after removing the smoothing and the post-ictal rule. Our threshold came from a per-fold ROC rule, which is not the paper's rule, so segment sensitivity and specificity cannot be compared one to one. The main point is that a working baseline now existed and everything after this builds on it.

D. Other Patients and a Pooled Set

The validation study in Section VII-D uses 23 CHB-MIT patients: chb01 to chb24 without chb21, which is the same subject as chb01. We use the same four channels, the same encoder, and the same windowing as for chb01. Each patient has its own train, validation and test files. The validation and

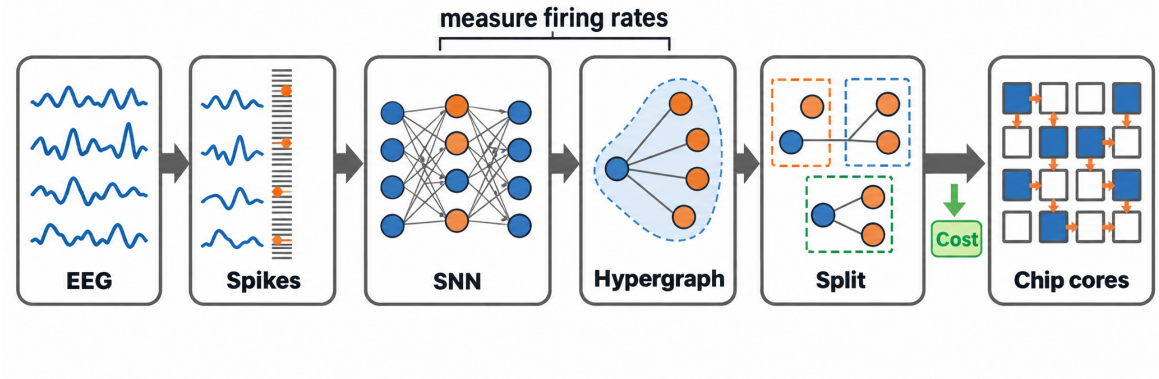


Fig. 1. Overall pipeline of this work, from EEG to the estimated cost on chip cores.

TABLE IV
EEG-SIDE ABLATION ON CHB01 (ONE SPLIT).

Setup	Sens.	Spec.	Epochs to converge
A: fixed 4 channels	0.989	0.991	9–12
B: data-driven 4 channels	0.968	0.995	9–12
C: hyperedge groups	0.968	0.990	12–15

TABLE V
FIRST MAPPING TESTS: INTER-CORE MESSAGES (KM1).

Network	Neurons/core	Naive	Hypergraph	Change
Real (73 neurons)	8	144.0	136.0	−5.6%
Real (73 neurons)	4	208.0	266.0	+27.9%
Synthetic (1000)	16	3752.4	2827.7	−24.6%
Synthetic (1000)	8	3911.8	2992.9	−23.5%

test files of a patient often hold very few seizure windows, so the AUC of a single patient is noisy. We also built one pooled set from the data of all patients.

IV. Early Ideas That Did Not Hold Up

A. Hypergraphs on EEG Channels

Our first hypothesis was that the baseline treats the four channels separately and misses how electrodes move together in a seizure. We tested three setups on chb01, changing only the channel strategy. Everything else stayed the same: encoder, network shape, training, and a split by file. Records 18 and 26 were held out.

- **A:** the four fixed channels of Busia *et al.*
- **B:** four channels picked from all 21 by a single-channel connectivity score.
- **C:** hyperedge groups. We computed coherence, cross-correlation, and mutual information for all channel pairs, subtracted the resting-state values, kept the strong pairs, joined them into groups, and averaged each group with connectivity weights.

Setup A won (Table IV). Setup C was not better than B and needed more epochs. There was also a warning sign in the construction itself. The threshold of the original method (mean plus one standard deviation) gave one giant group of 17 channels. We had to raise it to 1.75 standard deviations by hand to get four small groups. We decided not to tune it further, because tuning on one split would only fit that split. This is a single patient and a single split, so it does not prove that the idea can never work. It does show that the idea is not robustly better, so we dropped it as the main contribution. This fits the finding of Chung *et al.* [1], who also found on CHB-MIT that detectors with one or four well-chosen channels can work about as well as multi-channel ones, although their task and method differ from ours.

B. First Mapping Tests

After this, we moved the hypergraph to the hardware side. We took the trained 73-neuron network (64-8-1) and compared naive sequential partitioning with hypergraph partitioning (Mt-KaHyPar, KM1 objective). Because the real network was so small, we also built a synthetic sparse network of 1000 nodes with distance-decaying connections. Table V gives the number of messages that cross cores.

On the real network the result was small or even negative. We had predicted this before the run. Its first layer is fully dense, so every input connects to all 8 hidden neurons and there is no structure for a smart partitioner to find. On the synthetic sparse network the gain was about 24 percent at both core sizes. So the mechanism works when the network is sparse and local, but our trained network was too small and too dense to show it.

C. The Wearable Target Did Not Fit

A second problem is the wearable story. The design of Busia *et al.* [4] already uses 3271 of the 5280 logic cells (61%) on its FPGA. Its 529 weights fit in one core, so there is nothing to split. A bigger network would give the partitioner more to do, but it would also need more hardware than a wearable chip has. The sample period at 256 Hz is 3.9 ms (calculated here), while the reported inference time is 0.5 μs

(from [4]), so latency is not the bottleneck either. This does not mean that a full wearable system has no power or size limits. We therefore stopped treating the wearable as the goal. The question became: at what network size and sparsity does hypergraph mapping start to pay off for an EEG SNN?

D. Results We Later Withdrew

A first full pipeline gave a "90 percent sparse winner" with traffic ratios of 0.10 to 0.58. A careful review showed three problems. The sparse networks were not trained. The baselines never put an input neuron on the same core as its targets. And the sparse mask had planted groups of identical inputs, so finding them proved little. We withdrew the claim. The one result that held up was a null result: on the trained dense 73-neuron model, a simple rate-aware sequential mapping (traffic 1.48) beat hypergraph overlap partitioning (2.15).

A deeper attempt on a 417-neuron network was also inconclusive. The Mt-KaHyPar call crashed and the code silently fell back to a home-made annealer. The two methods called "EdgeMap" and "Overlap" turned out to be the same code. The headline network had a validation AUC of only 0.83 after 10 optimizer steps. Two useful facts still came out of it. A plain-graph count overestimated spike traffic by 6.9 to 31.5 times compared with the hypergraph count. And spreading the network over 99 cores instead of 14 raised energy from 167 to 287 pJ even though abstract traffic went down. We decided to stop patching this code and rebuild the pipeline from the paper.

V. Mapping Method

A. From a Trained SNN to a Hypergraph

After training, we run real EEG windows through the network and count spikes. The firing rate of a neuron is its spike count divided by the number of time steps and windows. We then build the hypergraph $H = (V, E)$ as in [9]. Each neuron is a node in V. Each neuron that sends spikes is the source $s(e)$ of exactly one hyperedge $e \in E$, and its targets $D(e) \subseteq V$ are the destinations given by the connection mask, so $e = \{s(e)\} \cup D(e)$. The weight w_e of a hyperedge is the measured firing rate of its source (Fig. 2). We use the firing rate because the cost we want to lower is the number of spikes that cross between cores, and a neuron that fires more often causes more of them.

One point cost us a lot of time early on. Spike frequency is not the synaptic weight. The learned weights decide which neurons are in a hyperedge. The firing rate of the source decides how heavy the hyperedge is. Rates must be measured by running data through the network. They cannot be read from the weight matrix. Input neurons count as nodes, and their spikes count as traffic, so the input rates always sum to 4.0.

B. Cost Model

Let λ_e be the number of different cores that hyperedge e touches, counting the core of the source. The connectivity cost is

$$C = \sum_{e \in E} w_e \, (\lambda_e - 1). \tag{1}$$

When a spike travels over d grid hops to another core, the energy and latency of that delivery are

$$E_d = d\,(E_R + E_T) + E_R, \qquad L_d = d\,(L_R + L_T) + L_R, \tag{2}$$

with $E_R = 1.7$ pJ, $E_T = 3.5$ pJ, $L_R = 2.1$ ns, and $L_T = 5.3$ ns. These values come from the Loihi-based table in [9], which bases them on Loihi [15]. The total energy is the weighted sum of E_d over all deliveries. The average latency is the weighted average of L_d. We call their product the energy-latency product (ELP), and lower is better. Every mapping is scored by the same function, so only the method changes and the yardstick does not. These numbers are estimates from a model, not measurements on a chip. A model like this shows which mapping moves fewer spikes over shorter paths. It leaves out effects such as memory limits inside a core and real router behaviour, so it cannot prove real energy savings on a chip.

C. Two Pipelines

We compare a naive pipeline with a hypergraph-aware one.

Naive. Neurons fill cores in the order of their IDs, which is layer by layer. Rates and wiring are not used. Cores are placed row by row on the grid. Sequential partitioning is also one of the simple methods studied in [9].

Hypergraph-aware. Each core starts with the busiest unassigned neuron. We then keep adding the neuron that shares the most inbound hyperedges with the core, until the core holds 32 neurons. This follows the idea of synaptic reuse, but it is a simpler greedy version and not a line-by-line copy of the algorithm in [9]. Cores are placed on the grid along a Hilbert curve. After that, we try 1500 random swaps of core positions and keep a swap only if the ELP gets better. The Hilbert curve start and the swap refinement follow the placement ideas of [9], [18]. The naive baseline is simple on purpose. It is a fair first control, but beating it does not show that we beat the stronger tools of Section II-E.

D. Ablation Design

The two pipelines differ in two ways at once: partitioning and placement. To find out which one matters, we ran four variants. Variant A is naive partitioning with row-by-row placement. B is naive partitioning with Hilbert placement and swap refinement. C is overlap partitioning with row-by-row placement. D is overlap partitioning with Hilbert placement and refinement, which is our full pipeline. The fair test of partitioning is D against B, because both get the same placement effort. We ran each variant over five seeds (42 to 46), since the swap step is random. The cost function is the same for all four variants, and B and D use the same 1500

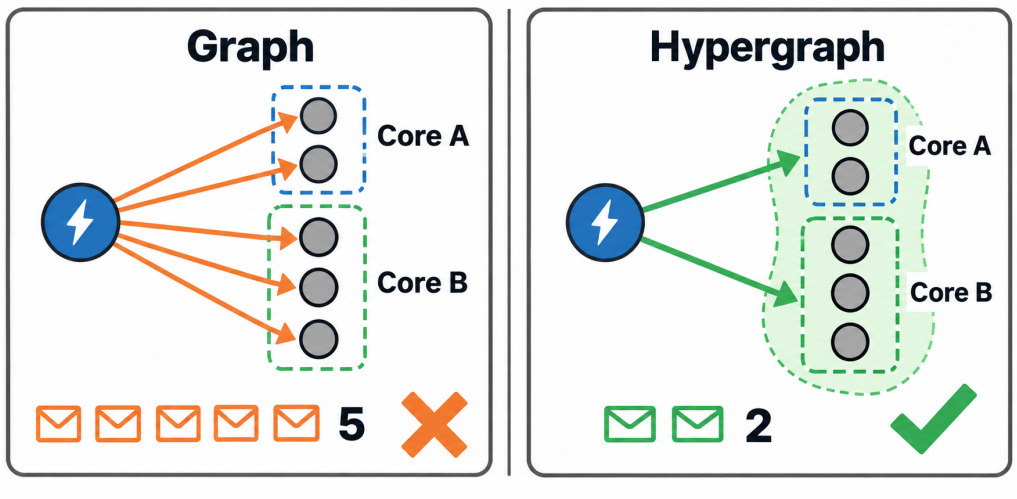


Fig. 2. A plain graph counts every connection as a separate message. A hypergraph counts one message per sending neuron and core.

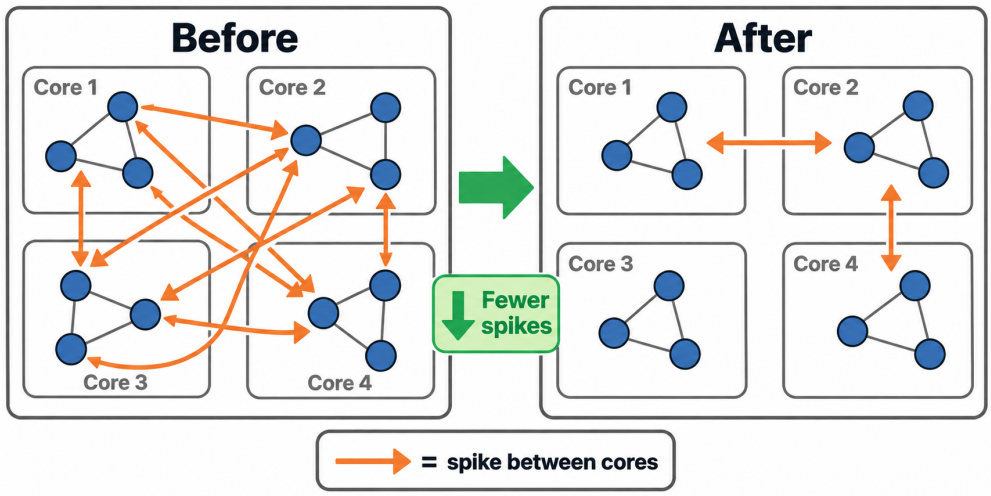


Fig. 3. The same neurons on four cores before and after hypergraph-aware mapping. Fewer spikes have to cross between cores.

swaps. The five seeds show how much the results move. They are not a significance test.

VI. Models and Experimental Setup

A. A Ladder of Six Models

We trained six SNNs in three sizes, each with a dense version and a 90 percent sparse version (Table VI). Sparse layers use a fixed fan-in mask, so every neuron keeps the same number of incoming synapses. The neurons are LIF with leak 0.9, threshold 1.0, reset by subtraction, and a fast-sigmoid surrogate gradient. The last unit is a non-spiking leaky integrator that is averaged over time, so it never sends spikes and carries no traffic. All models used the same recipe: Adam with learning rate 2×10^{-3}, batch size 64, five epochs (165 optimizer steps), binary cross-entropy, and gradient clipping at 1.0. The best epoch was picked by validation AUC.

B. Accuracy Gate

We decided on a gate before looking at any mapping result. A model is valid only if its output is not dead and both its validation and test AUC are at least 0.95. The threshold comes from the training data by Youden's J. Only valid models are mapped. We picked the value 0.95 before any mapping was run. The best epoch is chosen on the validation file only, but the test AUC is part of the gate, so the test file does decide which models get mapped. This rule is there because our earlier mapping results were measured on networks that did not work. The validation study in Section VII-D uses the gate differently, as a check and not as a filter (Section VI-D).

Five models passed. `S_sparse` failed, and we left it out and did not try to rescue it. At 90 percent sparsity, each of its 8 hidden neurons sees only 6 of the 64 inputs, and 33 input levels did not feed any neuron. Its AUC was below 0.5 on training and test data, so its score pointed the wrong way. We

TABLE VI
THE SIX MODELS AND THE ACCURACY GATE (AUC).

Model	Layers	Synapses	Val	Test	Gate
S_dense	64-8-1	520	1.000	1.000	pass
S_sparse	64-8-1	56	0.414	0.048	**fail**
M_dense	64-64-16-1	5136	1.000	1.000	pass
M_sparse	64-64-16-1	496	1.000	1.000	pass
L_dense	64-128-128-64-32-1	34848	1.000	1.000	pass
L_sparse	64-128-128-64-32-1	3488	1.000	1.000	pass

TABLE VII
NAIVE VS HYPERGRAPH-AWARE PIPELINE (SEED 42). GAINS ARE IN PERCENT. (PATIENT CHB01)

Model	Cores	ELP naive	ELP hyper.	ELP	Energy	Lat.
S_dense	3	501.6	344.4	31.3	4.6	28.0
M_dense	5	6908.5	2223.5	67.8	38.2	48.0
M_sparse	5	4557.7	1471.3	67.7	38.3	47.7
L_dense	14	43583.6	34332.5	21.2	11.1	11.3
L_sparse	14	33864.7	26734.0	21.1	13.5	8.7

think the missing inputs are the cause, but we did not test this.

An AUC of 1.0 on validation and test files that have 6 and 11 seizure windows shows that the models work. It does not show that they generalize. The raw encoded signal is also easy to separate. Single features, such as the spread of levels in a window, already give an AUC of 0.97 to 0.99. This is the likely reason that every working model reaches a near-perfect score. It also means that accuracy cannot tell our models apart, so we compare them on mapping cost.

C. Mapping Setup

We used 32 neurons per core. This is a deliberately small core, chosen so that a 417-neuron network has something to split. A real Loihi-style core is much bigger. The small setting in [13] holds 1024 neurons, 4096 axons, and 16384 synapses. The five valid models need 3, 5, 5, 14, and 14 cores. Cores sit on a square grid (4×4 for 14 cores). Firing rates were measured on the validation windows. We also built hypergraphs from the training windows, and the per-neuron rates of the two sets agree with a correlation of 0.990 to 0.998, so the choice of split is unlikely to change the findings. The hypergraphs have 73, 145, 145, 417, and 417 nodes. We checked that input rates sum to 4.000 and that synapse counts match the models. We also checked that each trained model reloads with the same output as saved (maximum difference zero on chb01; the check for the other patients is described in Section X).

D. Models for the Validation Study

For the validation study (Section VII-D) each patient has six models: three sizes (S, M and L, with the same architectures as in Table VI) and two variants. "Dense" is the trained fully connected model. "Pruned" is the dense model pruned to 90 percent sparsity with no fine-tuning afterwards, so some pruned models lost accuracy. These pruned models are not the fixed-mask sparse models of the chb01 study, which stay as they are. We call a pruned model "flagged" if it lost more than 0.05 AUC, on validation or on test, against its dense model. The verdict rule is the same rough screen as before. D vs B is YES if D is better than B by more than the sum of the standard deviations of B and D over the five seeds. It is NO if D is worse by more than that sum, and INCONCLUSIVE otherwise, which includes exact ties. PASS means the verdict is YES. FAIL means NO or INCONCLUSIVE. PASS is about partitioning and not about accuracy. For the three-core S models, D and B are often exactly identical (19 of the 46 S models), so INCONCLUSIVE there mostly means identical partitions and placements. All 144 models are mapped. The AUC gate of Section VI-B is not a filter here. We use it as a check (64 of the 138 patient models pass it). The pooled models are reported separately from the 23 patients. The mapping setup is the one above: 32 neurons per core, the same constants, five placement seeds, and 1500 swaps. All 144 models passed the reload check and the synapse-count check. Each model was trained with one seed.

VII. RESULTS

A. Hypergraph-Aware vs Naive Pipeline

Table VII compares the two full pipelines for a single run. The hypergraph-aware pipeline is better on modelled energy, latency, and ELP for all five models (Fig. 4 shows the same numbers as bars). Energy is the most careful number, with gains from 4.6 to 38 percent. ELP gains look bigger because ELP multiplies two reductions. All of these numbers are modelled, not measured.

A single run is noisy because the swap step is random, and single runs move by about 3 to 20 percent. Table VII shows one run (seed 42) so that every number can be traced, while the abstract uses the five-seed means. Over five seeds the mean ELP gains of the full pipeline are 31.3, 73.1, 70.0, 23.4, and 27.1 percent for the five models.

B. Where the Gain Comes From

Table VIII shows the ablation. Look first at B against A. Giving the naive partition the same Hilbert placement and refinement already brings most of the gain: 47.7, 76.3, 72.1, 20.1, and 18.1 percent. In other words, placement does most of the work. For the small and medium models, placement accounts for 100 percent or more of the total gain. For the two large models the share is 86 percent (L_dense) and 67 percent (L_sparse).

The fair test of partitioning is D against B. Here the answer depends on the model. On the small and medium models, overlap partitioning is neutral or worse. On L_dense it adds 4.1 percent, which only just passes our rough screen (the margin is larger than the sum of the two standard deviations). On L_sparse it adds 11.0 percent, and this one clearly passes. We think the reason is wiring. In a dense layer, every neuron talks to every neuron in the next layer, so any grouping looks the same to a partitioner. In the dense models the synaptic reuse is identical for both pipelines (29.42

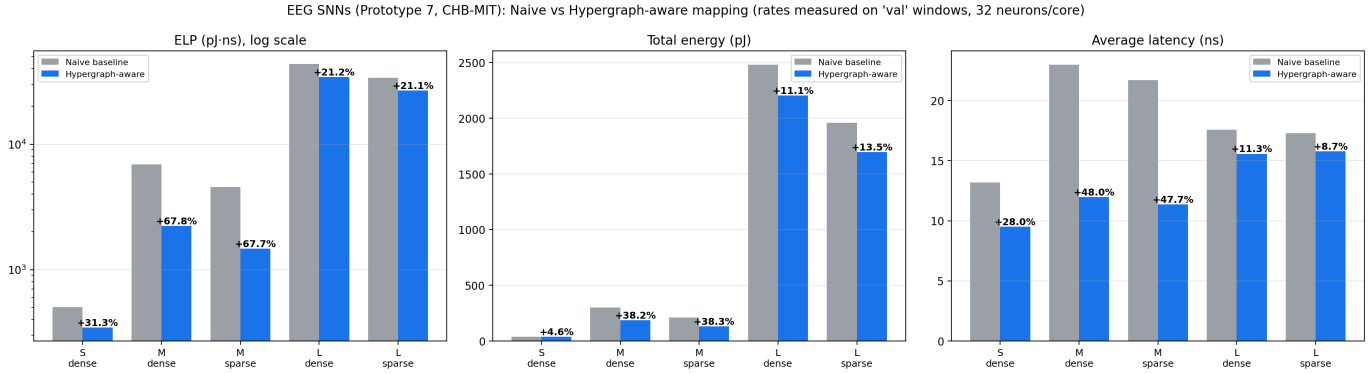


Fig. 4. Naive and hypergraph-aware mapping for the five valid models (single run, seed 42). Left: energy-latency product (log scale). Middle: total energy. Right: average latency. The percentage on each blue bar is the gain of the hypergraph-aware pipeline. (patient chb01)

TABLE VIII
ABLATION OVER FIVE PLACEMENT SEEDS. ELP VALUES ARE MEANS (LOWER IS BETTER); GAINS ARE IN PERCENT (POSITIVE IS BETTER). A: NAIVE + ROW PLACEMENT; B: NAIVE + REFINED PLACEMENT; C: OVERLAP + ROW PLACEMENT; D: OVERLAP + REFINED PLACEMENT. (PATIENT CHB01)

Model	ELP A	ELP B	ELP C	ELP D	D vs A (full)	B vs A (placement)	D vs B (partitioning)
S_dense	502	262	344	344	+31.3	+47.7	−31.3
M_dense	6909	1637	2483	1860	+73.1	+76.3	−13.6
M_sparse	4558	1269	2060	1367	+70.0	+72.1	−7.7
L_dense	43584	34821	45576	33404	+23.4	+20.1	+4.1
L_sparse	33865	27751	36717	24689	+27.1	+18.1	+11.0

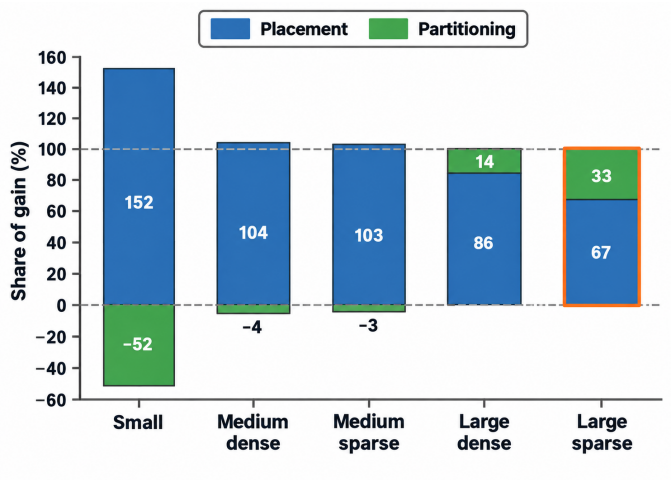


Fig. 5. Share of the total ELP reduction that comes from placement and from partitioning, for each model. (patient chb01)

for L_dense). Sparse wiring leaves different sets of shared inputs, so grouping can find something. This explanation fits the numbers, but we did not test it directly. It is an interpretation, not a tested cause.

Two more points are worth stating. First, C against A is mixed. On the large models, overlap partitioning with row-by-row placement is worse than naive (by 4.6 and 8.4 percent). We believe this is because the overlap method numbers its cores in the order it seeds them, which scatters communicating cores in a row layout, while naive core IDs follow the layers. We did not test this. It does suggest that the overlap partitioner needs good placement to pay off. Second, the placement step also uses the hypergraph cost (weighted traffic drives the swaps). So "mostly placement" does not mean the hypergraph did nothing. It means the hypergraph cost model pays off mainly at the placement step in these networks.

For S_dense, the head-to-head table shows a 31.3 percent gain, but the ablation shows that naive partitioning with refined placement is better still. Both statements are true. The pipeline beats naive, but placement is the reason, not partitioning.

C. Does It Repeat?

An earlier run used different trained models (four EEG models, one with a random 90 percent sparse mask). There the placement step also dominated, and overlap partitioning clearly helped only on the deep random-sparse model, by 13.7 percent. In the present run the same kind of model gives 11.0 percent. These are two separately trained sets of models, but both use the same patient and the same data, so this is a repeat of training and not a test on new data. We therefore say that the gain on the large sparse model appeared in both sets, and not that the whole finding is independently confirmed. What does not repeat is the sign for the dense deep model (within noise both times) and for the small model (0 percent then, −31 percent now). We note that the overlap partition of a 3-core network depends on exact firing rates, so it can change when the model is retrained. This was still the same patient, and the next subsection tests the finding on other patients.

D. Validation on 22 More Patients and a Pooled Model

A result from one patient could be a fluke. So we ran the same pipeline on 23 CHB-MIT patients, chb01 to chb24 without chb21, and on one pooled model. Patient chb01 is the first of the 23, and the other 22 are new. Each patient has six models (three sizes, dense and pruned), and the pooled set adds six more, so there are 144 models in total. The dense chb01 results are the same as in Table VIII, so they reproduce exactly. The pruned chb01 models are new, so the old sparse rows are not the same models. The data are described in Section III-D and the setup in Section VI-D.

1) Full Pipeline vs Naive: Table IX and Fig. 6(a) show the full pipeline against the naive one. D vs A is positive in all 138 patient models, from +15.2 to +78.6 percent. The group medians are about 47 to 49 percent for S, 66 to 70 for M, and 21 to 25 for L. Larger networks gain less from the full pipeline, which matches chb01. The six pooled models also all gain (+33.2 to +74.8 percent).

2) Where the Gain Comes From: Fig. 6(b) and (c), Fig. 7 and Table IX split the gain. Placement (B vs A) has medians of +47.7 and +51.5 percent for S, +75.9 and +65.9 for M, and +19.9 and +17.2 for L (dense, then pruned). Placement helps at every size.

Partitioning (D vs B) is a different story. Only `L_pruned` is clearly positive: median +9.3 percent, positive in 23 of 23 patients, and PASS in 14 of 23. `L_dense` has a median of -0.3, positive in 11 of 23, with PASS in 2. `M_dense` has a median of -22.9, positive in 1 of 23, with PASS in 1. `M_pruned` has a median of -2.6, positive in 11 of 23, with PASS in 10 of 23. This is mixed: partitioning helps in about half of the patients and hurts in the others. For S the median is -1.1 (dense) and 0.0 (pruned). In 19 of the 46 S models D and B are identical, which is why S is mostly a tie.

This matches the chb01 finding that placement does most of the work and partitioning helps only on the large sparse network. It does not match chb01 on two points. On `L_dense`, chb01 gave +4.1 and the patient median is -0.3. On the medium pruned networks, chb01 gave +7.1 and the patient median is -2.6, with mixed results. Fig. 7 shows the D vs B value of every model. The `L_pruned` row is green in all 23 patient columns. The `M_dense` row is not positive in 22 of 23. The S rows are mostly white (ties) or orange. The full per-model results are available as supplementary data.

We think wiring plays a role, since dense layers give any grouping the same traffic. But pruned models also fire less, so wiring and activity are mixed, and we did not separate them. Without chb01 the picture is the same (22 patients). `L_pruned` has a median of +8.8 with 22 of 22 positive and PASS in 13 of 22. `M_pruned` has a median of -3.7 (positive in 10 of 22, PASS in 9). `L_dense` has a median of -0.4 (PASS in 1 of 22), `M_dense` -23.4, `S_dense` -0.6, and `S_pruned` 0.0.

3) Pruned Models and Accuracy: Of the 69 pruned patient models, 40 are flagged (Table X). Table XI and Fig. 8 split the partitioning result by whether the pruned model kept its accuracy. Among the pruned models that kept it, `L_pruned` passes in 7 of 8 (median +13.6) and `M_pruned` in 6 of 8 (median +7.4). Among the flagged ones, `L_pruned` passes in 7 of 15 (median +7.3) and `M_pruned` in 4 of 15 (median -6.7). For S the counts are 1 of 13 and 0 of 10. The groups that kept accuracy have only 8 models each, so this is a lead, not a proof.

Pruned models also fire much less. Their firing rate is 35 to 63 percent of the rate of their dense models (mean ratio 0.352 for L, 0.463 for M, 0.629 for S), and the median ELP of the pruned model is 0.41 to 0.59 of the dense one (Fig. 9). So part of the lower cost of pruned models comes from lower activity and from damage. It is not a free gain.

4) Does the AUC Gate Change the Answer?: Table XII and Fig. 10 repeat the analysis for the 64 of 138 patient models that pass the gate. The full pipeline still beats naive in all 64. The partitioning medians are +15.3 for `L_pruned` (positive in 6 of 6, PASS in 5 of 6), +7.1 for `M_pruned` (PASS in 5 of 7), +1.6 for `L_dense` (PASS in 2 of 17), -20.6 for `M_dense`, -1.1 for `S_dense`, and -0.2 for `S_pruned`. The conclusions do not depend on the gate. The gated pruned groups are small (6 and 7 models), so their medians need care.

5) Pooled Model: Table XIII shows the pooled models. Pooled `L_dense` and `L_pruned` both gain +11.2 percent from partitioning (verdict YES). The pooled M models are inconclusive and the pooled S models do not gain. The three pooled pruned models are badly damaged (validation AUC 0.65 to 0.74, test AUC 0.59 to 0.67), and pooled `S_pruned` barely fires (its ELP of 73.8 for A is far lower than for any other S model). So these three need care. The pool is one model per size, so it supports the L result but is not a second sample.

Some things repeat. Placement helps at every size. Partitioning helps on the large pruned networks, in all 23 patients. Small networks give it nothing. Two things from chb01 do not repeat: the small gain on `L_dense` and the gain on the medium pruned networks. We did not test why the pruned large networks gain, and we did not test whether fine-tuned pruned models behave the same. We also did not train more than one seed per model. All of these numbers are model estimates, with one seed per model.

E. Hardware Verification

The formulas above assume an empty network, 32-bit floats, and only one core limit. We therefore ran four checks without changing any trained weight. We simulate the chip and do not measure a board. A development board draws hundreds of milliwatts of background power, while the on-chip network of `L_sparse` uses only a few microjoules for a whole 8 s window (4.4 to 5.2 μJ in our model). A wall-power meter could not see that difference. Fig. 11 gives an overview of the three main results below. These checks test different things. Quantization checks the model output with 8-bit arithmetic, the memory audit checks fit under limits we chose, the replay checks traffic under our routing model, and the Verilog run checks a few router counters. None of them is a measurement

TABLE IX

VALIDATION ON 23 PATIENTS: GAINS OF THE PIPELINES OVER THE NAIVE BASELINE, PER MODEL GROUP. VALUES ARE MEDIANS OVER THE 23 PATIENTS, IN PERCENT (POSITIVE IS BETTER); BRACKETS GIVE THE MINIMUM AND MAXIMUM. D VS B > 0 COUNTS PATIENTS WHERE PARTITIONING GAVE A POSITIVE GAIN. PASS COUNTS PATIENTS WHERE THE GAIN ALSO PASSED THE ROUGH SCREEN OF SECTION VI-D. THE POOLED MODEL IS IN TABLE XIII.

Group	n	D vs A, median [min, max]	B vs A (placement)	D vs B (partitioning)	D vs B > 0	PASS
S dense	23	+47.1 [+26.4, +54.1]	+47.7	−1.1	5/23	5/23
S pruned	23	+48.9 [+25.8, +63.7]	+51.5	0.0	1/23	1/23
M dense	23	+70.2 [+60.8, +78.6]	+75.9	−22.9	1/23	1/23
M pruned	23	+66.5 [+52.3, +72.9]	+65.9	−2.6	11/23	10/23
L dense	23	+21.5 [+15.2, +25.3]	+19.9	−0.3	11/23	2/23
L pruned	23	+24.9 [+17.3, +31.0]	+17.2	**+9.3**	**23/23**	**14/23**

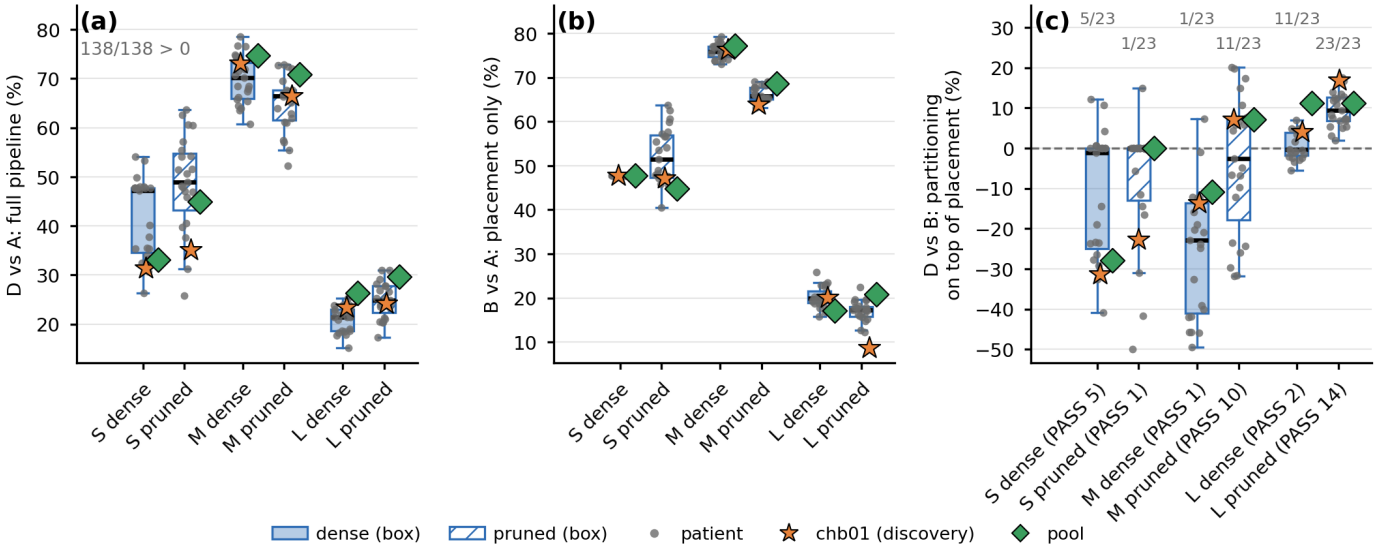


Fig. 6. Gains over the naive baseline for 23 patients, per model group. Each grey dot is one patient; boxes show the median and quartiles. (a) Full pipeline vs naive (D vs A). (b) Placement alone (B vs A). (c) Overlap partitioning on top of the same placement (D vs B). The orange star is chb01 and the green diamond is the pooled model.

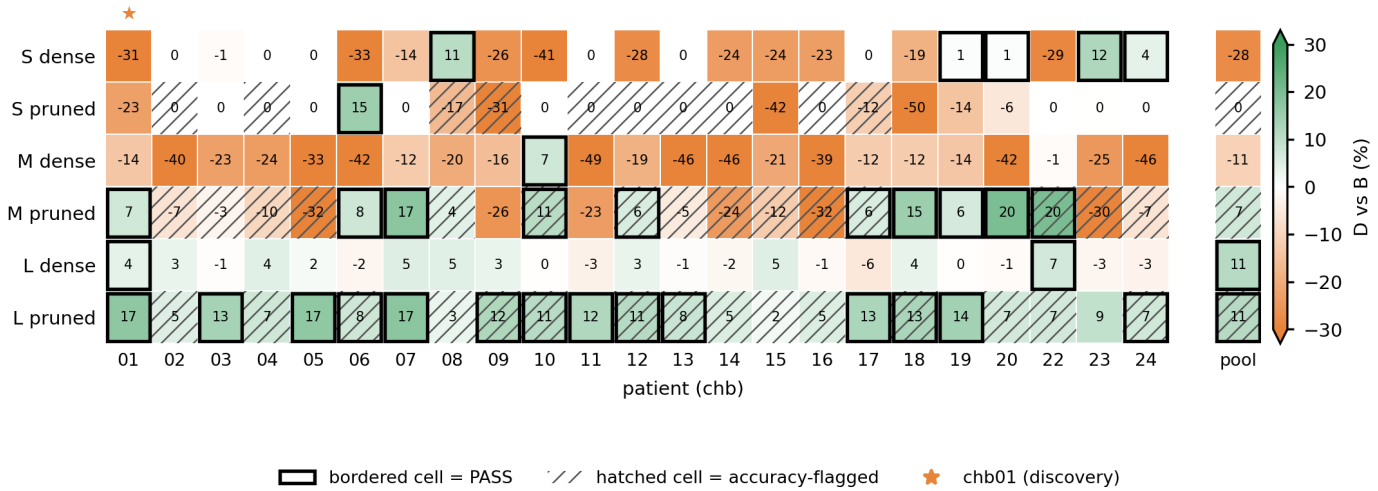


Fig. 7. D vs B gain (percent) of every model. Rows are the six model groups. Columns are patients and the pooled model. A thick border marks PASS; hatching marks pruned models that lost more than 0.05 AUC.

on a chip, and together they do not show that the models are ready to deploy. These checks were run on the chb01 models only.

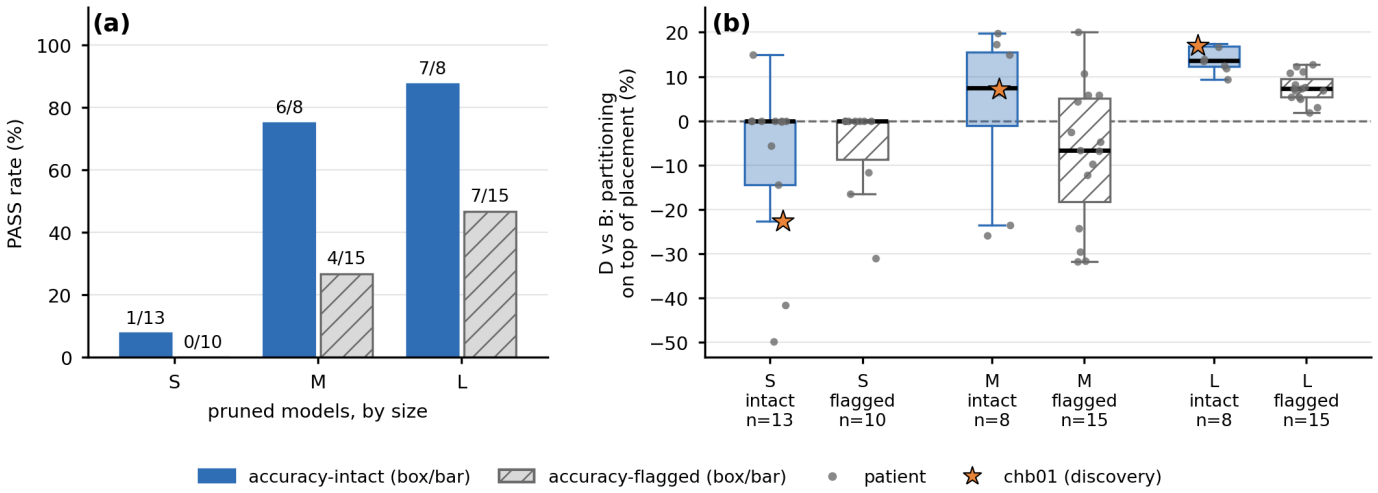


Fig. 8. Pruned models only, split by whether they kept their accuracy. (a) Share of models that pass; labels give passes over models. (b) D vs B gain for the same groups.

TABLE X

PRUNED MODELS AGAINST THEIR DENSE MODELS, FOR THE 23 PATIENTS (MEAN OR MEDIAN OVER PATIENTS). RATE RATIO IS THE FIRING RATE OF THE PRUNED MODEL DIVIDED BY THAT OF THE DENSE MODEL. ELP RATIO IS THE ELP OF THE PRUNED MODEL (PIPELINE D) DIVIDED BY THAT OF THE DENSE MODEL. ΔAUC IS THE CHANGE IN TEST AUC.

Size	Flagged	Rate ratio (mean)	ELP ratio (median)	ΔAUC (mean)	ΔAUC (median)
S	10/23	0.629	0.539	−0.103	−0.021
M	15/23	0.463	0.588	−0.206	−0.090
L	15/23	0.352	0.405	−0.081	−0.040

TABLE XI

PRUNED MODELS ONLY: PARTITIONING RESULT BY WHETHER THE PRUNED MODEL KEPT ITS ACCURACY. FLAGGED = LOST MORE THAN 0.05 AUC AGAINST ITS DENSE MODEL (VALIDATION OR TEST). MEDIAN D VS B IN PERCENT.

	Accuracy kept			Flagged		
Size	n	PASS	D vs B	n	PASS	D vs B
S	13	1	0.0	10	0	0.0
M	8	6	+7.4	15	4	−6.7
L	8	7	+13.6	15	7	+7.3

TABLE XII

SAME GROUPS, ONLY FOR MODELS WHOSE VALIDATION AND TEST AUC ARE BOTH AT LEAST 0.95 (64 OF 138 PATIENT MODELS). MEDIANS IN PERCENT.

Group	n	D vs A	D vs B	D vs B > 0	PASS
S dense	9	+47.1	−1.1	3/9	3/9
S pruned	11	+46.9	−0.2	0/11	0/11
M dense	14	+71.1	−20.6	1/14	1/14
M pruned	7	+67.3	+7.1	5/7	5/7
L dense	17	+22.6	+1.6	9/17	2/17
L pruned	6	+25.5	+15.3	6/6	5/6

1) 8-bit quantization: We turned the weights into signed 8-bit integers by scaling each layer's largest weight to 127.

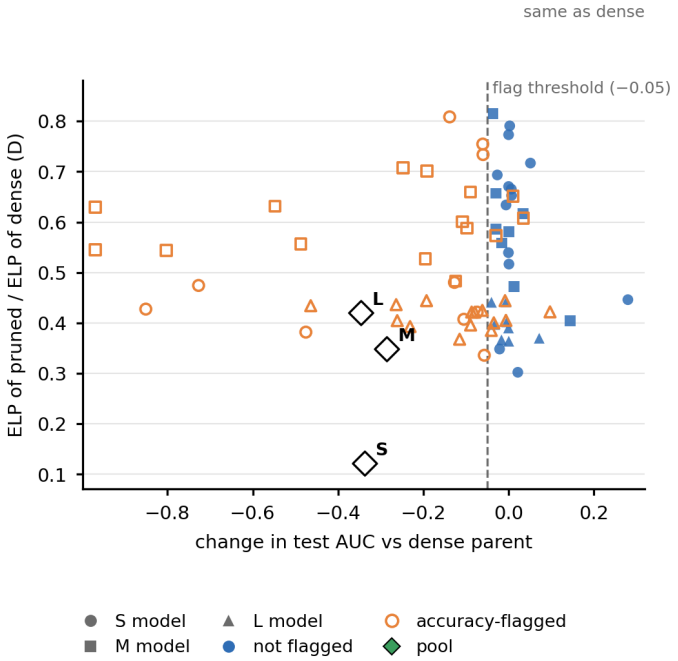


Fig. 9. Cost against accuracy for the pruned models. x: change in test AUC against the dense model. y: ELP of the pruned model divided by ELP of its dense model. Hollow markers lost more than 0.05 AUC. Lower cost comes together with accuracy loss, so it is not a free gain.

Membrane values use 16-bit integers, and the leak of 0.9 becomes a multiply by 230 followed by a right shift of 8 ($230/256 \approx 0.898$). All five models kept an AUC of 1.000 on validation and test data, with zero loss. Weight memory dropped by 75 percent (Table XIV).

2) Core memory audit: Until now we only enforced the neuron limit. We now check all three limits with scaled values of 32 neurons, 128 inbound axons, and 512 synapses per core. These follow the 1:4:16 ratio of the paper. Table XV shows the

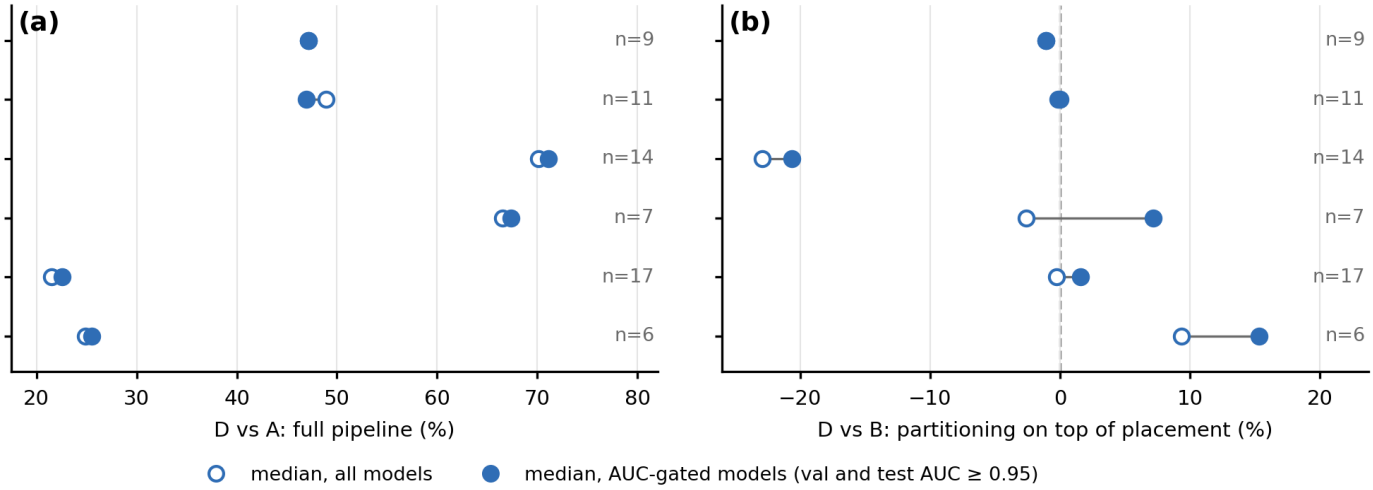


Fig. 10. Effect of the AUC gate. Hollow circle: median over all models of the group. Filled circle: median over models with validation and test AUC both at least 0.95 (n at the right). (a) D vs A. (b) D vs B.

TABLE XIII
THE POOLED MODELS (ONE MODEL PER SIZE AND VARIANT). ELP IN PJ·NS. A DAGGER MARKS PRUNED MODELS THAT LOST MORE THAN 0.05 AUC AGAINST THEIR DENSE MODEL (ALL THREE DID). GAINS IN PERCENT.

Model	Cores	ELP A	ELP B	ELP D	D vs A	D vs B	Verdict	Val AUC	Test AUC
S dense	3	501.6	262.2	335.3	+33.2	−27.9	NO	0.912	0.924
M dense	5	7300.6	1661.3	1841.8	+74.8	−10.9	INCONCLUSIVE	0.960	0.952
L dense	14	40956.1	33961.9	30158.5	+26.4	+11.2	YES	0.970	0.942
S pruned†	3	73.8	40.7	40.7	+44.9	0.0	INCONCLUSIVE	0.648	0.587
M pruned†	5	2197.1	689.9	641.1	+70.8	+7.1	INCONCLUSIVE	0.688	0.667
L pruned†	14	17991.7	14249.0	12659.5	+29.6	+11.2	YES	0.736	0.596

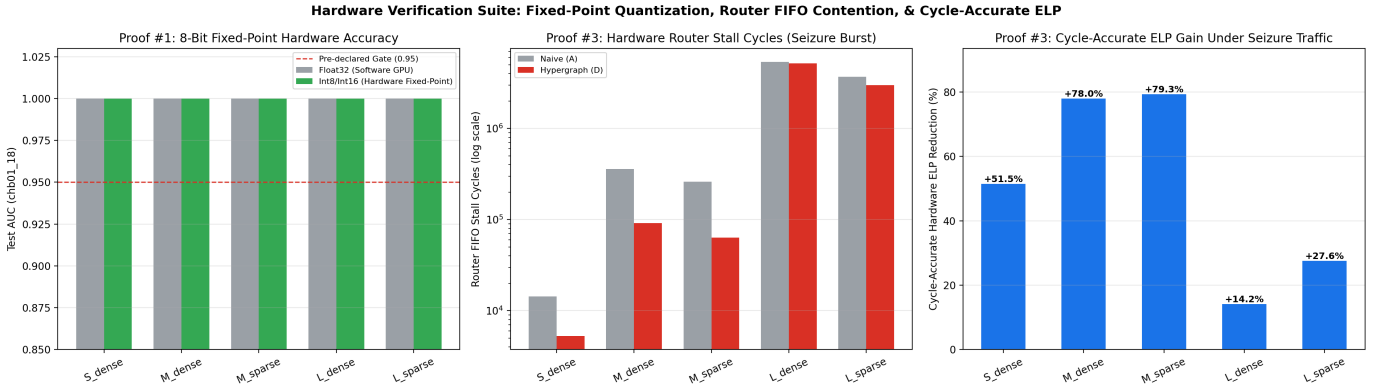


Fig. 11. Hardware verification results. Left: test AUC stays at 1.0 after 8-bit quantization for all five models (the dashed line is the 0.95 gate). Middle: router buffer stall cycles in the seizure window, log scale. Right: cycle-accurate ELP gain of the hypergraph-aware pipeline in the seizure window. (patient chb01)

result. The axon limit was met everywhere. The synapse limit is where models fail. Both dense models with more than one layer (`M_dense` and `L_dense`) overflow by a factor of 4 to 8. Both sparse models fit. For `S_dense`, the naive partition needs 520 synapses in one core and overflows by 8, while the overlap partition needs exactly 512 and fits. For `L_sparse`, overlap partitioning cuts the total number of inter-core axon table entries from 1078 to 971 (9.9 percent less). Note that these limits are chosen by us, so the audit shows the effect of sparsity and does not prove that any specific chip would accept the models.

3) Cycle-accurate mesh replay: Our formulas use average firing rates, and 98 to 99 percent of the windows are calm EEG. A seizure is rare, but it makes many neurons fire at once, and this is when the chip is under the most stress. We built a trace-driven simulator in the style of [10], [11]. We replayed the real spike trains of the held-out file (record 18) through a 2D mesh with X-then-Y routing, one step at a time.

TABLE XIV
QUANTIZATION. AUC IS THE SAME BEFORE AND AFTER FOR EVERY MODEL. (PATIENT CHB01)

Model	Val AUC	Test AUC	FP32 (KB)	INT8 (KB)
S_dense	1.000	1.000	2.03	0.51
M_dense	1.000	1.000	20.06	5.02
M_sparse	1.000	1.000	1.94	0.48
L_dense	1.000	1.000	136.12	34.03
L_sparse	1.000	1.000	13.62	3.41

TABLE XV
CORE MEMORY AUDIT. N = NAIVE, O = OVERLAP. LIMITS: 32 NEURONS, 128 AXONS, 512 SYNAPSES PER CORE. (PATIENT CHB01)

Model	Max synapses/core	Axon table entries	Fits (N / O)
S_dense	520 / 512	64 / 48	no / yes
M_dense	2048 / 2048	192 / 208	no / no
M_sparse	192 / 192	176 / 181	yes / yes
L_dense	4096 / 4096	1120 / 1120	no / no
L_sparse	416 / 416	1078 / 971	yes / yes

At each of the 2048 steps, every spiking neuron sends one packet to each other core that holds a target. If more than one packet wants the same link in the same step, the extra packets wait in a buffer. In our model, each wait adds 2.1 ns and 0.45 pJ. We replayed one normal window and one seizure window.

Table XVI gives the results, and there are three things to read from it. First, the number of packets for L_dense is exactly the same in both pipelines. In a dense network, a neuron must reach every core that holds the next layer, whatever the grouping. For L_sparse, overlap partitioning removes 14,523 packets in the seizure window, and the packet count is a pure partitioning effect. Second, on the small and medium models the hypergraph-aware pipeline actually sends more packets, but over much shorter paths (average hops per packet fall from about 2.7 to 1.3 on M_sparse). This is a placement effect, and it removes most of the stalls. Third, L_sparse gains more during the seizure window (27.6 percent ELP, 16.6 percent dynamic energy, and 691,758 fewer stall cycles) than in the normal window (13.5 percent). Since this replay compares naive partitioning and placement against the full pipeline, it cannot separate the two parts, and it covers one window of each kind.

4) Verilog router simulation: Finally, we wrote Verilog for the 2D XY-router mesh of L_sparse in two versions, one with the naive mapping tables and one with the hypergraph-aware tables. We simulated the first 256 seizure clock cycles at 100 MHz in Icarus Verilog and also ran the logic through Yosys synthesis. We do not report area or timing numbers here. The hardware counters in Table XVII agree with the Python simulator on the size of the reduction.

This is a simulation of the routing and counting logic for 256 cycles, not a finished chip. What it does show is that the lookup tables and the routing counts can be written as hardware, and that the numbers match the software model.

VIII. INTERACTIVE DASHBOARD

Tables and bar charts are hard to follow for people who do not work with neuromorphic chips. So, after the experiments were done, we built a small web page that brings the work of the earlier sections into one place and lets a person watch it run. This dashboard is not a new result and it is not a finished product. It is a visualisation of what we have done so far. It uses the same trained model, the same hypergraph, the same two mappings, and the same cost formulas as the earlier sections. Only the packet replay and the stall counter were written fresh for the page, and we describe them below.

A. What the Dashboard Does

Python runs once and writes a set of static files. The web page only plays these files back, so it needs no server and it runs offline on a laptop. It is written in plain JavaScript with the PixiJS drawing library [23]. The page replays the whole hour of record 18, the held-out test file, as 450 windows of 8 s. The model is M_sparse (145 neurons and 496 synapses), placed on five cores of a 4×4 grid. The viewer can pick the naive pipeline, the hypergraph-aware pipeline (seed 42), or both side by side. Fig. 12 shows the whole page.

B. The Panels

The page has five parts.

- **EEG and score strips.** The four EEG channels scroll across the top. Below them, a score line shows the model output for each 8 s window, with the alarm threshold as a dashed line. When a window scores above the threshold, a soft alarm banner appears.
- **The network.** All 145 neurons are drawn in layers, and each one is coloured by the core where it lives. A neuron glows briefly when it spikes.
- **The chip.** A 4×4 mesh shows the five used cores and the eleven unused tiles. Small dots travel between cores along X-then-Y routes, one dot for each message.
- **The counters.** They show packets, hops, energy, latency, ELP, the gain against naive, and stalls.
- **The timeline.** It shows the score of the whole hour and a slider. The seizure annotation is hidden by default, so a viewer can watch the detector find the seizure on its own. A button jumps to 30 s before the annotated onset.

C. Seeing the Two Mappings

Fig. 13 shows the network and the chip for each pipeline. In the naive mapping, neurons fill the cores in ID order, so each core holds neurons of mostly one layer and the network looks striped. The five cores sit in a row. In the hypergraph-aware mapping, each core starts from a busy neuron and grows by shared inbound hyperedges, so neurons from different layers share a core and the colours mix. The cores sit along a Hilbert curve and are then refined by swaps. This is the same idea as Fig. 3, but now drawn for a real trained network with real firing rates.

TABLE XVI
CYCLE-ACCURATE REPLAY OF ONE SEIZURE WINDOW OF RECORD 18 (N = NAIVE, H = HYPERGRAPH-AWARE). "STALLS CUT" IS THE DROP IN BUFFER STALL CYCLES. ELP GAINS ARE SHOWN FOR THE NORMAL WINDOW AND THE SEIZURE WINDOW. (PATIENT CHB01)

Model	Packets N	Packets H	Hops N	Hops H	Stalls cut (%)	ELP gain normal (%)	ELP gain seizure (%)
S_dense	8,192	8,965	12,288	8,965	63.0	+27.4	+51.5
M_dense	35,045	43,105	97,476	59,357	74.5	+82.3	+78.0
M_sparse	31,047	36,590	84,712	46,485	75.6	+75.6	+79.3
L_dense	342,792	342,792	717,077	622,819	3.3	+15.0	+14.2
L_sparse	280,926	266,403	597,309	494,383	18.8	+13.5	+27.6

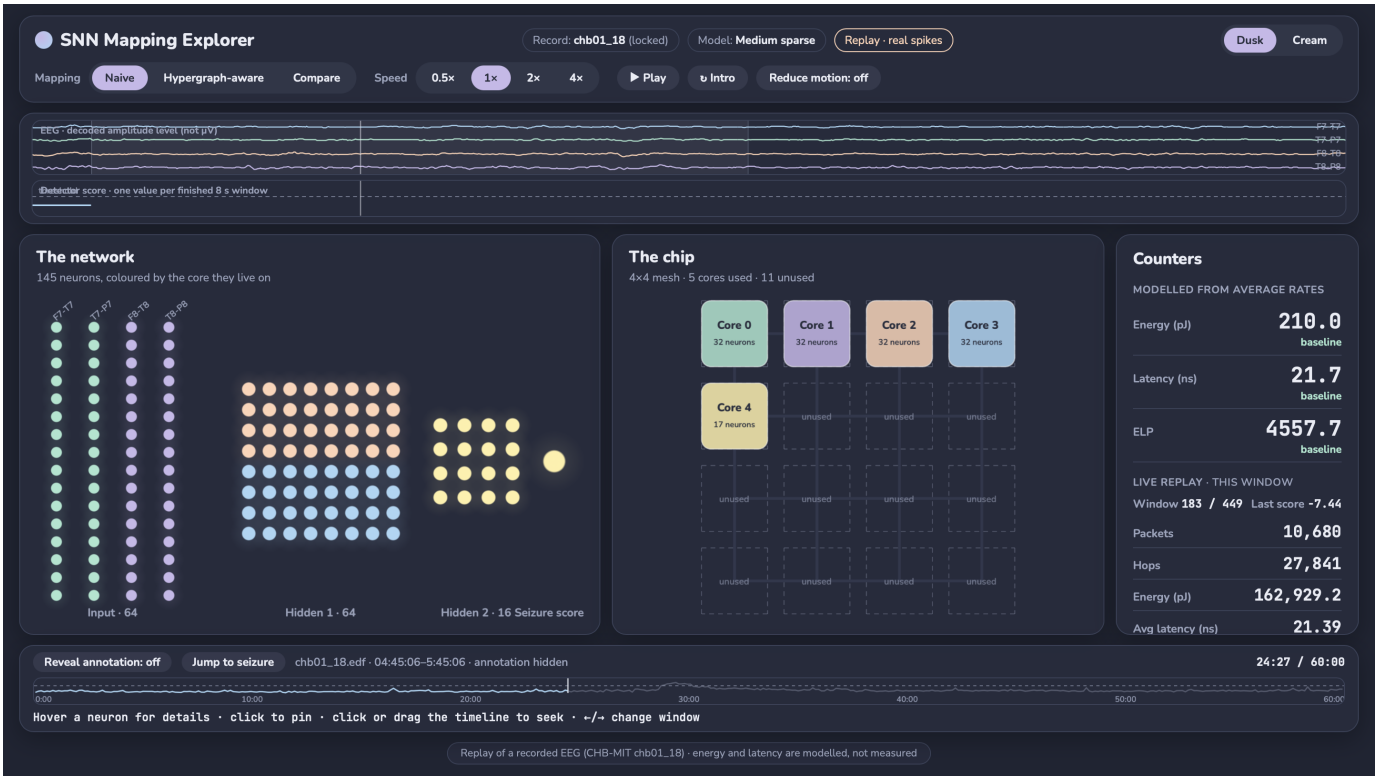
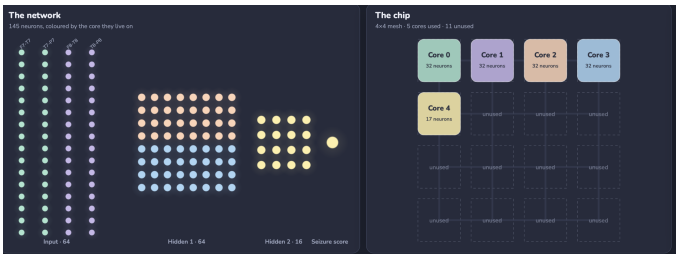
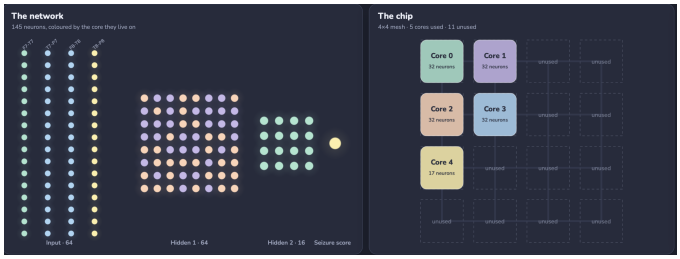


Fig. 12. Overview of the dashboard. The top shows the four EEG channels and the score strip with the alarm threshold. The middle shows the network on the left, the simulated chip on the right, and the live counters. The bottom shows the whole recorded hour as a timeline that the viewer can drag.



(a) Naive mapping



(b) Hypergraph-aware mapping

Fig. 13. The network and the chip in the dashboard for the two pipelines. (a) Naive mapping: neurons fill cores in layer order, so the colours form layer stripes and the cores sit in a row. (b) Hypergraph-aware mapping: neurons from different layers share cores, so the colours mix, and the cores are placed by the Hilbert curve and swaps.

D. Hovering Over a Neuron

The most useful part for teaching is the hover. When the viewer points at a neuron, all other neurons fade and the hyperedge of that neuron is drawn as a fan to its targets. On the chip, the cores that receive the spike light up. A small label compares two counts: a plain graph would count one message for each connection, and the hypergraph counts one message for each destination core. This is the idea of Fig. 2,

TABLE XVII
VERILOG SIMULATION OF THE L_SPARSE ROUTER MESH, FIRST 256 SEIZURE CYCLES AT 100 MHZ. (PATIENT CHB01)

Counter	Naive	Hypergraph	Change
Packets	34,407	32,616	-5.2%
Mesh hops	73,724	60,220	-18.3%
Router energy (pJ)	441,856	368,591	-16.6%

shown on a real neuron of the trained model. Fig. 14 shows one neuron hovered in each mapping. The wiring of the neuron is the same in both. Only the cores differ, so the number of destination cores, and with it the number of messages, can differ.

E. Live Comparison During a Seizure

In compare mode, the chip panel splits in two. One chip shows the naive mapping and the other shows the hypergraph-aware mapping, and both are fed by the same spikes. The counters then show two columns and the difference between them (Fig. 15). When the seizure starts, many neurons fire at once, so the traffic on the chip becomes visibly denser. The counters use the real spikes of record 18 and the same energy and latency formulas as Section V-B.

F. What the Dashboard Does Not Claim

We kept the page honest on purpose, and we state the limits here.

- It is a replay of a recorded EEG, not live patient monitoring. It is one patient (chb01), one record (18), and one model (`M_sparse`). We do not claim that it generalises.
- Energy and latency are modelled with the Loihi-based constants of Section V-B. They are not measured on a chip.
- The mapping was built from average firing rates on the validation file, but the counters use the real spikes of record 18. So the live totals are not equal to the formula ELP in Table VII. The export script checks that the formula values for `M_sparse` are reproduced (ELP 4557.7 for naive and 1471.3 for hypergraph-aware, seed 42), and it stops if they are not.
- The stall counter is a simple demo model, where each link carries at most one packet per step and extra packets count as stalls. It is not a replacement for the replay of Section VII-E, and its numbers should not be compared with Table XVI.
- A window is scored only after its 8 s have passed. So in the replay the alarm is raised after window 215 is complete, which is about 8 s after the annotated onset at 1720 s. It is not instant.
- We did not test whether the page helps people understand the work. That would need a study with viewers.
- The page compares only the full hypergraph-aware pipeline with the naive pipeline. It does not show the split between placement and partitioning from Section VII-B, because that result is better read in a table.

IX. WHAT WE LEARNED

This section is the main point of the paper. Each result below changed what we did next. Table XVIII lists the full chain, and the paragraphs after it explain the reasoning.

A. Lessons

1. More EEG channels did not mean more information. In our setup, grouping many channels into hyperedges gave a lower sensitivity and slower training than four fixed channels. Our guess is that the extra channels added noise, and we did not test this. We also felt it was wrong to keep tuning the threshold until the result looked good. We dropped the idea instead. This is a result on one patient and one split. It rules out "clearly better", but not "might help somewhere".

2. A wearable chip and a mapping study pull in opposite directions. A wearable wants the smallest possible network. Mapping only pays off when there is a network large enough to split. Once we saw this, we stopped designing for a wearable and asked a different question: for which sizes and wiring does hypergraph mapping help? Mapping also cannot change accuracy. It only changes where neurons live. So accuracy is a check, not a goal.

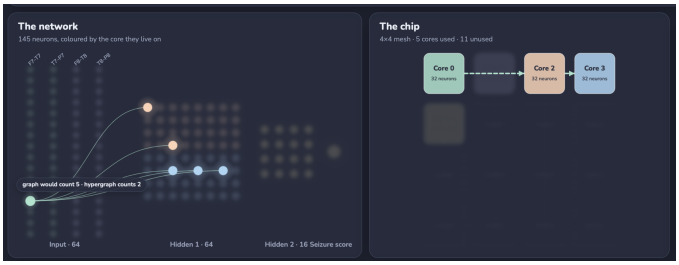
3. A small dense network gives a partitioner nothing to do. In a dense layer every neuron sends to every neuron in the next layer, so any grouping has the same traffic. We saw this three times: the null result on the 73-neuron model, identical synaptic reuse on `L_dense`, and identical packet counts on `L_dense` in the mesh replay. This is a statement about our cost model and our tested networks, not a general law. Across 23 patients the dense large models gave a median D vs B of -0.3 percent, and the dense medium models -22.9 percent, so the partitioner still did nothing useful on dense networks (this is again a statement about our cost model and tested networks).

4. A comparison is only as good as its baseline. Our first big wins came from baselines that never put inputs next to their targets. After we fixed the baseline and used one yardstick for all methods, the wins shrank or disappeared. We now treat any large early win as a reason to check the baseline.

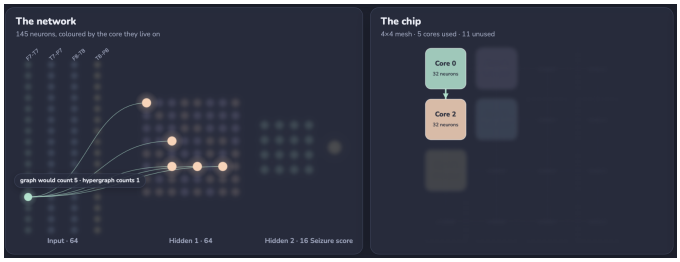
5. Fix the model before judging the mapping. When a network scores 0.83 or 0.70 AUC, its firing rates say little about a seizure detector. So we set an accuracy gate in advance and used a separate test file. A model that fails the gate (`S_sparse`) is out. We do not rescue it. In the validation study we mapped every model and used the gate only as a check, because per-patient AUC is noisy when a split holds few seizure windows. The conclusions did not depend on the gate.

6. A broken tool is not a failed idea. The crashed Mt-KaHyPar call and the silent fallback taught us two things. A silent fallback hides what you are really testing. And a result from a tool that did not run is not evidence for or against the method. After that we allowed no hidden fallbacks.

7. Rebuild small, prove it on controlled data, then move to real data. Patching the old pipeline made it hard to tell a bug from a finding. Writing the code again from the paper,



(a) Hover, naive mapping



(b) Hover, hypergraph-aware mapping

Fig. 14. Hovering over one sample neuron. The other neurons fade, the hyperedge of the chosen neuron is drawn to its targets, and the receiving cores light up on the chip. (a) Naive mapping. (b) Hypergraph-aware mapping. The same neuron sends its spike to a different set of cores in each mapping.

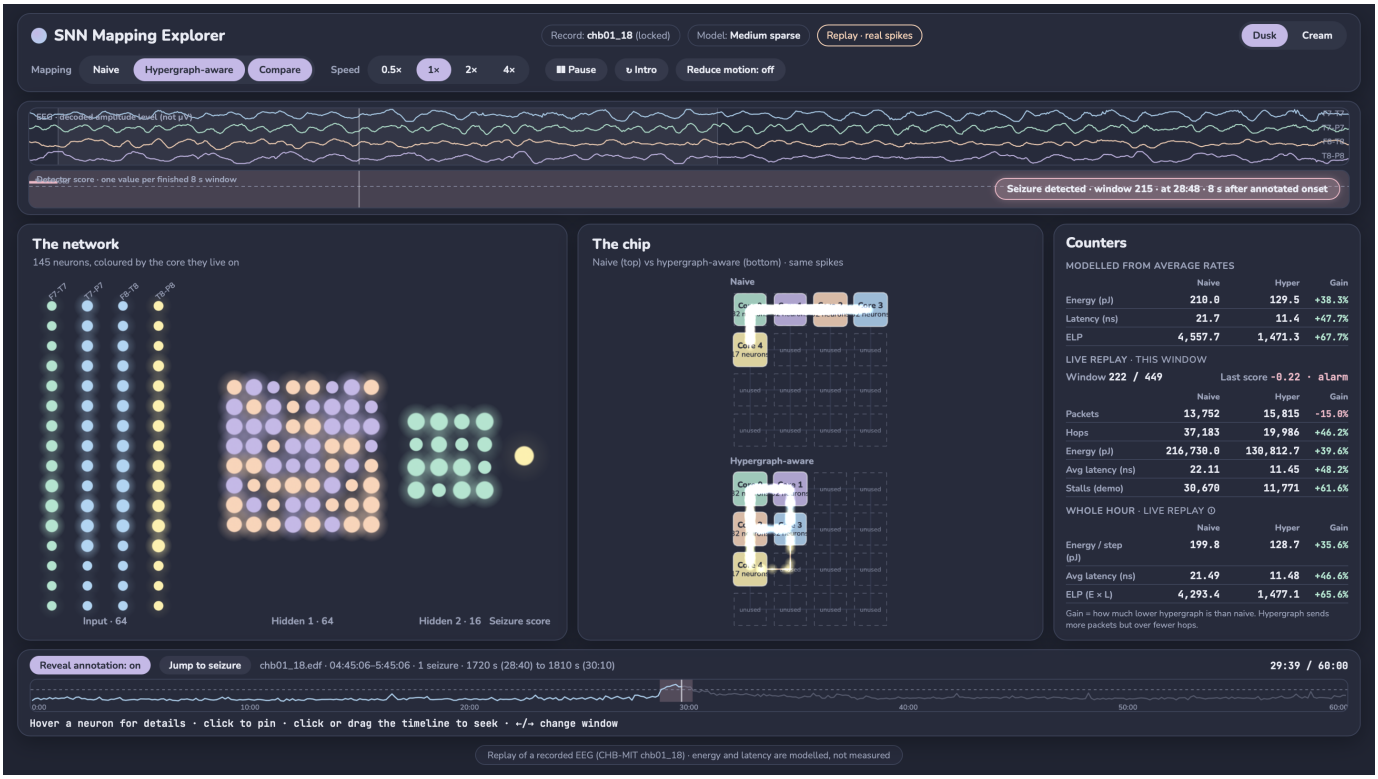


Fig. 15. Live comparison during the seizure. Both chips receive the same spikes of record 18. The counters on the right show packets, hops, energy, latency, ELP, and stalls for the naive and the hypergraph-aware pipeline.

and testing it first on synthetic data where every model trains well, gave a pipeline we could trust and explain.

8. Ask where a gain comes from. The first headline was "hypergraph pipeline wins by 7 to 48 percent". The ablation changed this to "placement wins, and partitioning adds a clear gain only on a large sparse network". The second sentence is less exciting, but it is what the data say, and it repeated on a second set of models (13.7 percent, then 11.0 percent) and again on 22 more patients (for the large pruned networks).

9. Sparsity decides whether a model fits and whether partitioning matters. Dense multi-layer models overflowed the core memory by 4 to 8 times. Sparse ones fit. Dense models also gave partitioning no packets to remove. So the benefit of this mapping is tied to sparse wiring, and sparse SNNs are also the ones that are practical for hardware. On 23 patients, partitioning gained on large pruned networks and not on large dense ones. But pruned models also fire less and often lose accuracy, so wiring and activity are mixed together here.

10. Seizure traffic is the case that matters. Averaged over mostly calm EEG, `L_sparse` gained 13.5 percent ELP in the replay. During the seizure window it gained 27.6 percent. A mapping that is tuned on average traffic may look modest but still help most when the chip is busiest. Because we tested one window of each kind, we count this as a lead and not a proof.

11. Report the negative results. Several of our most useful findings were negative: the EEG-side hypergraph, the dense-network null result, the failed model, and the placement-versus-partitioning split. They show what to try and what not

TABLE XVIII
DECISION LOG: WHAT WE TRIED, WHAT HAPPENED, WHAT WE CONCLUDED, AND WHAT WE DID NEXT.

Step	What we tried	What happened	What we concluded	What we did next
1	Replicate the wearable 64-8-1 detector	AUC 0.998 and event sensitivity 1.0, but more false alarms and one dead fold	A working baseline exists. Training budget and post-processing matter	Used it as the base for all later work
2	Group EEG channels into hyperedges before encoding	Fixed 4 channels won (sens. 0.989). Groups needed more epochs and gave no gain	More channels did not help here, and the grouping needed a hand-tuned threshold	Dropped it. Moved the hypergraph to the hardware side
3	Map the real 73-neuron model and a synthetic 1000-node network	-5.6% or worse on the real model, about -24% messages on the synthetic one	The idea needs sparse structure. Our real model was too small and dense	Planned bigger, sparser, trained models
4	Check the wearable fit	529 weights fit in one core, and 61% of the FPGA logic is already used	Tiny network: nothing to split. Big network: too large for a wearable chip	Dropped the wearable as the goal. Asked when mapping pays off
5	First full pipeline with a "90% sparse winner"	Win came from untrained nets, weak baselines, and a planted mask	A comparison is only as good as its baseline	Withdrew the claim. Same metric for every method
6	Deeper model with hypergraph code (Prototype 6)	Inconclusive: failed accuracy gate, 10 steps, crashed tool, duplicate code	The failure was in tools and design, not in the idea	Stopped patching. Rebuilt from the paper
7	Rebuild on synthetic data, then move to EEG models	ELP gains 33, 21, 34% (synthetic) and 48, 44, 20, 7% (EEG). Ablation: mostly placement	Most of the gain is placement. Two models were too weak to trust	Retrained all models with a pre-declared gate and a test file
8	Six-model ladder with the gate (Prototype 7)	Five passed, `S_sparse` failed. Partitioning helps on the large sparse model (+11.0%)	The pipeline wins everywhere. Partitioning matters only for large sparse wiring	Checked hardware realism
9	Quantization, memory audit, mesh replay, Verilog	No loss at 8 bits. Dense models overflow. Seizure stalls cut. Verilog agrees	Sparsity decides whether a model fits. Seizure traffic shows the biggest gain	Built a dashboard to show the work
10	Put the model, the two mappings, and the cost model into a web page (`M_sparse`, record 18)	The page shows the network, the chip, and the traffic, and lets a viewer hover, compare, and seek through the hour	The mapping is much easier to follow when it can be seen. The page is a visualisation, not a finished product	Checked the main result on more patients
11	Run the same pipeline on 22 more patients and a pooled model (144 models, all mapped, AUC gate used as a check)	Full pipeline beat naive in 138 of 138 patient models. Partitioning gained only on large pruned networks (median +9.3%, positive in 23 of 23), not on small ones, mixed on medium ones	Placement always helps. Partitioning is a large-network effect. Pruned models look cheaper partly because they fire less and lose accuracy	Wrote up. Listed the missing tests (Section X)

to try.

12. One patient can mislead. On chb01 `L_dense` just passed our screen (+4.1 percent). Over 23 patients its median is -0.3 percent and it passes in 2 of 23. A pattern should be checked on other patients before we trust it.

13. A cheaper model is not a better model. Pruned models have an ELP of 0.41 to 0.59 times that of their dense models (median), but they fire at 35 to 63 percent of the dense rate and 40 of 69 lost accuracy. Cost must be read together with accuracy.

B. The Answer We Reached

We ended with a more modest question than the one we started with: for an EEG-based SNN, at what size and sparsity does hypergraph mapping pay off? Our data give a conditional answer. Traffic-aware placement, which uses the hypergraph cost, helps at every size. On the 23 patients, partitioning helps on the large pruned networks (median 9.3 percent, positive in all 23, passes in 14 of 23). It is mixed on the medium pruned ones, and it does not help on small networks or on dense ones. The pruned models are cheaper partly because they fire less, so we do not read this as a free gain. A seizure detector that already works at 73 neurons fits one small core, so the regime where partitioning pays off is larger than what a wearable detector needs. We think this is an honest result, and it points to what to test next.

X. LIMITATIONS

These points limit how far the results can be pushed.

- **Few seizures per patient, one seed.** The chb01 study and the 22 other patients use one split each, with few seizure windows per file. For chb01 the validation and test files each hold a single seizure event (6 and 11 positive windows). Per-patient AUC is noisy, and each model was trained with one seed. An AUC of 1.0 shows that the models work, not that they generalize. The encoded signal is also easy to separate, so a perfect AUC may say more about the data than about the model.
- **Pruned models are not like-for-like.** They are prune-only with no fine-tuning. Many lost accuracy, and they fire less than dense ones, so dense and pruned ELP are not a like-for-like comparison.
- **The pooled result is one model per size.** It is not a second sample.
- **Hardware checks and dashboard use chb01 only.** The other 22 patients were scored with the analytical cost model only.

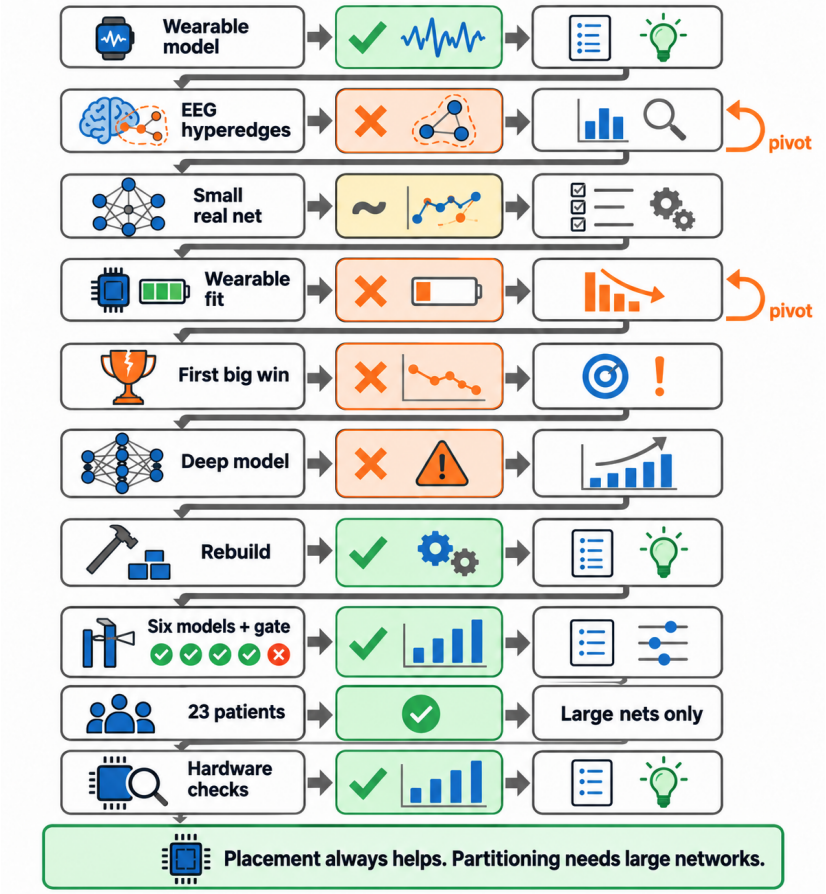


Fig. 16. How the result of each experiment guided the next one. Green means the idea worked, orange means it failed.

- **The reload check is not exact for the other patients.** For chb01 the reload gave the same output (maximum difference zero). For the other patients and the pooled models, we compared AUC, firing rates and silent counts, not exact logits, because of float differences across GPU stacks.
- **Axon and synapse limits are only audited.** They are checked after partitioning (Section VII-E) and are not used inside the partitioner, which is where synaptic reuse should matter most.
- **Few baselines.** The final study uses one naive baseline. It does not compare a graph model against a hypergraph model, and Mt-KaHyPar never ran in the final pipeline. So our claim is "hypergraph-aware pipeline versus naive pipeline", not "hypergraph model versus graph model". Our overlap partitioner is a simpler greedy version of the published algorithm. We did not run SpiNeMap, SNEAP, EdgeMap, or the hypergraph tools of Ronzani and Silvano on our models.
- **Placement search is simple.** It only swaps core positions with 1500 random trials, and it never moves a core to an empty grid cell. Our verdict rule (margin larger than the sum of two standard deviations) is a rough screen and not a significance test. Five seeds show how much the results move, but they are too few for a proper test.
- **Estimates, not measurements.** Energy and latency come from a model with Loihi-based constants and small 32-neuron cores. They cannot be compared with the 4.55 nJ of the wearable FPGA. The 32-neuron cores are our choice and do not describe any real chip.
- **Rates come from mostly calm EEG.** Per-class rates (normal vs seizure) were not studied separately.
- **The replay is narrow.** It uses one normal and one seizure window of one record of patient chb01, and it compares full pipelines, so it cannot separate placement from partitioning.
- **The Verilog model is small.** It covers the router and counting logic of one network for 256 cycles. We do not report area or timing.
- **The dashboard is a visual aid.** It shows one model (`M_sparse`) on one record of one patient, as a replay and not as live monitoring. Its counters are modelled, its stall counter is a simple demo model, and we have not tested it with other viewers or on other patients. It is not a clinical tool and not a finished product.
- **Short training.** Models trained for 165 steps. The earlier replication used 10 epochs instead of several hundred.

XI. Conclusion and Future Work

We studied how to map seizure-detecting SNNs onto neuromorphic cores using a hypergraph model. On five models that passed an accuracy gate, a hypergraph-aware pipeline lowered the estimated energy-latency product by 23 to 73 percent against a naive layer-order pipeline. An ablation showed that most of this gain comes from traffic-aware placement. On chb01, hypergraph partitioning added a clear gain only on the large sparse network. A similar gain appeared on the large pruned networks of 22 more patients, and it also appeared in two sets of trained models of chb01. The models lost no accuracy at 8 bits, the 90 percent sparse models fit our core memory limits while the dense ones did not, and a cycle-accurate replay and a Verilog router model supported the gain, with the largest effect in a seizure window. These findings do not yet show energy savings on a real chip, and the hardware checks cover one patient only.

To make these results easier to follow, we also built a small web dashboard. It replays a recorded EEG hour through one trained model and shows the network, the chip, and the traffic for both pipelines. It is a visualisation of the work in this paper, not a finished product.

We also checked the main result on 22 more patients and on a pooled model, 144 models in all. The accuracy gate was used only as a check there. The full pipeline beat the naive one in 138 of 138 patient models. Partitioning gained only on the large pruned networks (positive in all 23 patients, and clearly passing our screen in 14 of 23). Pruned results must be read together with accuracy, because those models fire less and many lost accuracy.

The path to this result was not straight. The EEG-side hypergraph idea and the wearable target both failed, and some of our early wins were withdrawn. Each failure removed one wrong explanation, and this is why the final result can be trusted.

Next we plan to do the following.

- Use the axon and synapse limits inside the partitioner, and add a plain-graph baseline, runs of earlier tools such as those in Table I, and a working Mt-KaHyPar run on the same cost function.
- Implement the published overlap algorithm closely, and improve the placement search (moves to empty cells, real annealing, more seeds, and a proper significance test).
- Test topologies where node order no longer follows data flow, such as recurrent and skip connections, as a study of when the method helps and not as a way to make it win.
- Replay more windows and measure per-class rates; run the hardware checks and the dashboard on more patients; train more than one seed per model; fine-tune the pruned models and try other sparsity levels.
- Extend the dashboard to other models (such as `L_sparse`) and other patients, and put it online.
- Move from simulated mesh cost to a multi-core FPGA design with measured resource use.

References

[1] Y. G. Chung, A. Cho, H. Kim, and K. J. Kim, "Single-channel seizure detection with clinical confirmation of seizure locations using CHB-MIT dataset," *Front. Neurol.*, vol. 15, Art. no. 1389731, 2024, doi: 10.3389/fneur.2024.1389731.

[2] J. D. Nunes *et al.*, "Spiking neural networks: A survey," *IEEE Access*, vol. 10, pp. 60738–60764, 2022, doi: 10.1109/ACCESS.2022.3179968.

[3] D. Kudithipudi *et al.*, "Neuromorphic computing at scale," *Nature*, vol. 637, no. 8047, pp. 801–812, 2025, doi: 10.1038/s41586-024-08253-8.

[4] P. Busia, G. Leone, A. Matticola, L. Raffo, and P. Meloni, "Wearable epilepsy seizure detection on FPGA with spiking neural networks," *IEEE Trans. Biomed. Circuits Syst.*, vol. 19, no. 6, pp. 1175–1186, Dec. 2025, doi: 10.1109/TBCAS.2025.3575327.

[5] A. H. Shoeb and J. V. Guttag, "Application of machine learning to epileptic seizure detection," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 975–982.

[6] J. Guttag, "CHB-MIT scalp EEG database (version 1.0.0)," PhysioNet, 2010, doi: 10.13026/C2K01R.

[7] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000, doi: 10.1161/01.cir.101.23.e215.

[8] A. N. Pisarchik, N. Peña Serrano, W. Escalante Puente de la Vega, and R. Jaimes-Reátegui, "Hypergraph analysis of functional brain connectivity during figurative attention," *Appl. Sci.*, vol. 15, no. 7, Art. no. 3833, 2025, doi: 10.3390/app15073833.

[9] M. Ronzani and C. Silvano, "A case for hypergraphs to model and map SNNs on neuromorphic hardware," arXiv:2601.16118, 2026.

[10] A. Balaji *et al.*, "Mapping spiking neural networks to neuromorphic hardware," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 1, pp. 76–86, 2020, doi: 10.1109/TVLSI.2019.2951493.

[11] S. Li *et al.*, "SNEAP: A fast and efficient toolchain for mapping large-scale spiking neural network onto NoC-based neuromorphic platform," in *Proc. ACM Great Lakes Symp. VLSI (GLSVLSI)*, 2020, pp. 9–14, doi: 10.1145/3386263.3406900.

[12] J. Xue *et al.*, "EdgeMap: An optimized mapping toolchain for spiking neural network in edge computing," *Sensors*, vol. 23, no. 14, Art. no. 6548, 2023, doi: 10.3390/s23146548.

[13] M. Ronzani and C. Silvano, "GPU-accelerated hypergraph partitioning and placement to map SNNs on neuromorphic hardware," arXiv:2609.07577, 2026.

[14] J. K. Eshraghian *et al.*, "Training spiking neural networks using lessons from deep learning," *Proc. IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023, doi: 10.1109/JPROC.2023.3308088.

[15] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018, doi: 10.1109/MM.2018.112130359.

[16] S. Song *et al.*, "DFSynthesizer: Dataflow-based synthesis of spiking neural networks to neuromorphic hardware," *ACM Trans. Embed. Comput. Syst.*, vol. 21, no. 3, Art. no. 27, 2022, doi: 10.1145/3479156.

[17] C. Xiao, J. Chen, and L. Wang, "Optimal mapping of spiking neural network to neuromorphic hardware for edge-AI," *Sensors*, vol. 22, no. 19, Art. no. 7248, 2022, doi: 10.3390/s22197248.

[18] O. Jin *et al.*, "Mapping very large scale spiking neuron network to neuromorphic hardware," in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, vol. 3, 2023, pp. 419–432, doi: 10.1145/3582016.3582038.

[19] Q. Xing *et al.*, "SNNcut: An efficient partitioning method for large-scale spiking neural networks using spike-sharing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, 2025, doi: 10.1109/TCAD.2025.3648616.

[20] Ü. Çatalyürek *et al.*, "More recent advances in (hyper)graph partitioning," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 253, 2023, doi: 10.1145/3571808.

[21] L. Gottesbüren *et al.*, "Scalable high-quality hypergraph partitioning," *ACM Trans. Algorithms*, vol. 20, no. 1, Art. no. 9, 2024, doi: 10.1145/3626527.

[22] A. Balaji *et al.*, "Enabling resource-aware mapping of spiking neural networks via spatial decomposition," *IEEE Embedded Syst. Lett.*, vol. 13, no. 3, pp. 142–145, Sep. 2021, doi: 10.1109/LES.2020.3025873.

[23] PixiJS contributors, "PixiJS: The HTML5 creation engine." [Online]. Available: https://pixijs.com